

KVault — Panel Brief

A kernel-backed encrypted secrets vault with role-based access control. Dripta Pandit · 6,559 lines of C and C++17 · Linux 7.1.5+kali-amd64

Full detail in [EXPLANATION.md](#); design in [docs/03-design.md](#).

1. The problem

Applications keep secrets in config files or environment variables. Any process running as that user can read them, and nothing records who did. Encrypting the file does not help — the application must decrypt it, so the key has to be reachable by the application; the problem moves rather than goes.

Two weaknesses: **granularity**, since the smallest protectable unit is a whole file, so services sharing a user can read each other's credentials; and **accountability**, since a config read is an ordinary file read, indistinguishable from a compromised process reading it for the first time.

2. What KVault does

Three things move into the kernel:

The key	Derived from a passphrase in user space, handed over once, wiped immediately. Lives only in kernel memory; destroyed on seal, on an inactivity timer, and on module unload.
The decision	A <i>reference monitor</i> in ring 0 authorises every operation from <code>current_uid()</code> — read by the kernel from the caller's own credentials, not from anything the caller sends.
The record	Every attempt is audited, allowed or denied, written by the same code path that takes the decision, before the result is returned.

The result behaves like a software TPM: a device that holds a key, works with it for callers it recognises, and logs what it was asked.

3. Architecture

USER SPACE (ring 3)	<code>vaultctl · kvsim · kvsetup → libkvault (VaultClient, KeyDeriver, SecureBuffer, SealedStore, PolicyLoader, AuditViewer)</code>
KERNEL (ring 0)	<code>ioctl(/dev/kvault) — read+poll(/dev/kvault_audit)</code> <code>kv_main (cdev, 2 minors, sessions, 13 ioctls, auto-lock timer)</code> <code>kv_acl ← THE REFERENCE MONITOR kv_crypto (gcm(aes))</code> <code>kvault.ko kv_store · kv_file · kv_audit (ring + wait queue) · kv_proc</code> vault state: key[32], KCV, salt, counters — the key's only home

Two minors because there are two audiences: `/dev/kvault` (group `kvault`, 0660) takes commands; `/dev/kvault_audit` (group `kvaudit`, 0440) is a read-only stream. The auditor is in `kvaudit` only and cannot open the control device at all — file permissions outside, the reference monitor inside.

4. Five decisions worth defending

Decision	Why
AES-256-GCM, not CBC	Tamper detection is a requirement. CBC gives confidentiality only; detecting modification would mean bolting on a MAC and getting the composition right. GCM authenticates in the same pass.
The secret's name is GCM associated data	Without it, an attacker who can edit the vault file swaps <code>db_password</code> 's ciphertext under <code>public_motd</code> 's name — every tag still verifies, and a principal allowed to read the second is served the first. Binding the name makes the swap fail.
A fresh random nonce on every write	Nonce reuse does not weaken GCM, it collapses it: the XOR of both plaintexts leaks, and so does the authentication key. A counter is cheaper and wrong — restoring a backup rewinds it.
crypto_memneq, not memcmp	<code>memcmp</code> returns at the first differing byte, leaking how many leading bytes were right — enough, over many attempts, to recover the value byte by byte.
memzero_explicit, not memset	The compiler may delete a <code>memset</code> whose result is provably never read, which is exactly the case when wiping before a free.

5. Evidence it works

Check	Result
Unit / integration / system suites	33 / 19 / 4 — 56 automated cases, 0 failed, 0 skipped
Multi-user simulation	16/16 steps matched policy; 14 audit records, 7 denials
RBAC + persistence session	26/26
Fuzz (400 iterations, random names and lengths)	No undocumented errno, no crash
Concurrency	8 processes × 50 reads; 6 threads × 40 mixed ops — zero mismatches
Tamper detection	One flipped bit → EBADMSG; the running vault untouched
Kernel health	No WARNING, BUG, lockdep or KASAN; rmmod clean
Performance	Full GET round trip 1.3 µs; 2.9 GB/s at 4 KiB (generic-gcm-vaes-avx2)

The simulation is the proof (docs/screenshots/04-kvsim.png): kv_alice and kv_bob hold the **same role** and get different answers — the role is not the decision, the ACL is. kv_eve is refused four times by the kernel; kv_carol at open() by file permissions.

6. The defect worth asking about

Six defects were found during development. **None by reading the code.** The one that matters: a guest with no permissions could run PUT on a name that did not exist and become its **owner** — and owners have full control.

The attack is name-squatting: claim a name an application expects to own, and its writes are refused (denial of service) or it stores a secret you can read.

It survived review because *every individual check in PUT was correct*. The missing one answers a question ACLs structurally cannot: they attach to an object, and at creation time there is none. Nor could it fold into the role defaults, which are the fallback for *existing* secrets — a default WRITE for developers would have granted write access to every secret in the vault. The fix is a separate predicate, kv_role_may_create(). The simulator found it the first time it ran a complete scenario.

7. The two questions worth rehearsing

Why the kernel and not a daemon? The boundary is hardware-enforced — no user-space process can ptrace ring 0 or read its memory, whereas a daemon holding a key is a process like any other. Identity needs no protocol: current_uid() is what the kernel already knows, not a claim to verify. And there is nothing to bypass — the device is the only path to the ciphertext. The cost, plainly: a bug here is a kernel bug, which is why the module does as little as possible and validates every byte crossing the boundary.

What stops root reading secrets? Nothing — and claiming otherwise would be dishonest. Root can load a module, read kernel memory, or unload mine. The threat model names this: the adversary is a local, unprivileged or *differently*-privileged process. The honest claim is that the bar moves from "any process of the same user" to "ring 0", with a record of everything tried. Hardware-backed storage is the answer when root is in scope.

GCM vs CBC, memzero_explicit, and the trustworthiness of current_uid() are covered by §4 above; fifteen further questions are answered in EXPLANATION.md §19.

8. Stated limits

Root is out of scope (above). PBKDF2 at 200,000 iterations raises the cost of an offline attack on an exported vault; it does not rescue a weak passphrase. The software-path crypto benchmark could not be obtained — aesni_intel is in use here and rmmod refuses. kmemleak was not enabled; it needs a reboot. Secret names and owner UIDs are visible in the vault file by design: names must be readable to index entries without decrypting everything.